Kasi Yedumati · Full Stack Training

Module: Variables, Datatypes, Operators, and I/O

Phase: Core Java
Previous module: Java Setup and First Programs
Next module: Flow Control

Module Outcome:

- Declare a labelled box (`String studentName = "Ravi"`) with the right type
- Do money and marks maths — and print a line a user can read
- Stay **linear** today: no `if` / loops yet, just data moving through `main`

Teaching-order: programs in this module stay **linear** — no `if` / `while` / `for` / extra methods. Decisions and loops start in Flow Control.

What is a variable?

A **variable** is a named box in memory that holds a value. The label is `studentName`; the contents are `"Ravi Kumar"`.

🏦 Bank 📱 UPI 👤 Employee 💰 Payroll 📦 Order

Where used?
- **HDFC Bank** / **PhonePe** — balance, transfer amount
- College apps — roll, marks, fees
- **Employee Management Portal** — basic salary, HRA
- **Swiggy** — line price × qty, GST totals
- **Apollo Hospitals** — patient age, ticket amount, `isActive` flags

Java is **statically typed**: you declare the type. You cannot put `"hello"` into an `int` box.

```
String studentName = "Ravi Kumar";
int rollNumber = 101;
double percentage = 87.5;
boolean isPass = true;
```

Primitive types (core)

Type	Example	Use in training
int	`42`, `-7`	Counts, roll (if numeric), qty
long	`9999999999L`	Big whole numbers
double	`99.99`	Rupees, percentages, GST
boolean	`true`, `false`	Flags
char	`'A'`	One character (grade letter)

Type	Example	Use in training
String	`"Priya"`	Names, emails (not a primitive)

Prefer **`double` for money in this course** (simple). Banks use `BigDecimal` later — label that **Later — not this week**.

Type conversion (casting)

`Scanner.nextLine()` and `next()` give **text**. Convert for math.

```
int age = Integer.parseInt("22");
double price = Double.parseDouble("49.50");
```

Narrowing needs a cast:

```
double marks = 78.6;
int truncated = (int) marks;   // 78 — fractional part dropped
```

Common Error: `Integer.parseInt("78.5")` throws `NumberFormatException`. Use `Double.parseDouble` for decimal marks.

Operators — quick map

```
Arithmetic:  +  -  *  /  %
Comparison:  ==  !=  <  >  <=  >=
Logical:     &&  ||  !
Assignment:  =  +=  -=  *=  /=
```

`int / int` is **integer division** (`5 / 2` is `2`). Use a `double` for rupees.

Input and output

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
String name = sc.nextLine();
System.out.println("Hello, " + name);
sc.close();
```

`+` between a string and a number **concatenates**. `"Total: ₹" + total`

Important Concepts and Definitions

Term	Definition
Variable	Named storage with a declared type
Literal	Value written in code: `10`, `"hi"`, `true`
Primitive	Built-in value type (`int`, `double`, …)

Term	Definition
Casting	Convert type: `(int) 3.9` or `Integer.parseInt`
Operator	Symbol that computes (`+`, `==`)
Operand	Value an operator uses
Scanner	Class that reads keyboard / text input
Wrapper	Object form of a primitive (`Integer` for `int`)
Concatenation	Joining with `+` when a `String` is involved
Modulo (`%`)	Remainder after division

Step-by-Step Notes (With Annotated Code)

Step 1: Variables for a student record

```
public class StudentRecord {
    public static void main(String[] args) {
        int roll = 2026001;
        String name = "Priya Nair";
        int marksJava = 78;
        int marksSql = 82;
        double average = (marksJava + marksSql) / 2.0;

        System.out.println("Roll: " + roll);
        System.out.println("Name: " + name);
        System.out.println("Average marks: " + average);
    }
}
```

Expected Output:

```
Roll: 2026001
Name: Priya Nair
Average marks: 80.0
```

Note:
- Wrong: `(marksJava + marksSql) / 2` stays `int` if both are `int`.
- Prefer: divide by `2.0` so the result is `double`.

Store Ravi's roll `2026002`, Java marks 81, and SQL marks 74, then print his name, roll, and average.

```
public class RaviAverage {
    public static void main(String[] args) {
        String roll = "2026002";
        String name = "Ravi";
        int javaMarks = 81;
        int sqlMarks = 74;
        double average = (javaMarks + sqlMarks) / 2.0;
        System.out.println(name + " (" + roll + ") average: " + average);
```

```
    }
}
```

Expected Output:

```
Ravi (2026002) average: 77.5
```

Store food ₹185.50 and travel ₹60, add them, and print today's total with ₹.

```
public class DailyExpense {
    public static void main(String[] args) {
        double food = 185.50;
        double travel = 60.0;
        double total = food + travel;
        System.out.println("Today's total: ₹" + total);
    }
}
```

Expected Output:

```
Today's total: ₹245.5
```

Step 2: Datatypes and `char` / `boolean`

```
public class EmployeeLite {
    public static void main(String[] args) {
        String employeeId = "E1024";
        int yearsExperience = 3;
        double ctcLakh = 4.5;
        boolean isRemote = true;
        char band = 'B';

        System.out.println(employeeId);
        System.out.println(yearsExperience);
        System.out.println(ctcLakh);
        System.out.println(isRemote);
        System.out.println(band);
    }
}
```

Expected Output:

```
E1024
3
4.5
true
B
```

Store a PhonePe ID `priya@okaxis` and a transfer of ₹120.00 as `double`, then print both.

```
public class UpiAmount {
    public static void main(String[] args) {
```

```
        String upiId = "priya@okaxis";
        double amountInr = 120.00;
        System.out.println(upiId);
        System.out.println(amountInr);
    }
}
```

Store `isLoggedIn` as false and `hasPaidFees` as true, then print both flags.

```
public class Flags {
    public static void main(String[] args) {
        boolean isLoggedIn = false;
        boolean hasPaidFees = true;
        System.out.println(isLoggedIn);
        System.out.println(hasPaidFees);
    }
}
```

Step 3: Arithmetic operators (canteen)

```
public class CanteenMath {
    public static void main(String[] args) {
        double price = 40.0;
        int qty = 2;
        double gstRate = 0.05;
        double subtotal = price * qty;
        double total = subtotal * (1 + gstRate);

        System.out.println("Subtotal: ₹" + subtotal);
        System.out.println("Total: ₹" + total);
        System.out.println("Remainder demo 10 % 3 = " + (10 % 3));
    }
}
```

Expected Output:

```
Subtotal: ₹80.0
Total: ₹84.0
Remainder demo 10 % 3 = 1
```

Remember: `5 / 2` is `2`. `5 / 2.0` is `2.5`.

Print `5 / 2` (integer division) and `5 / 2.0` (double division) on two lines.

```
public class DivisionDemo {
    public static void main(String[] args) {
        System.out.println(5 / 2);
        System.out.println(5 / 2.0);
    }
}
```

Expected Output:

```
2
2.5
```

Split a ₹450 bill three ways and print what each person pays.

```
public class SplitBill {
    public static void main(String[] args) {
        double bill = 450.0;
        double each = bill / 3.0;
        System.out.println("Each pays: ₹" + each);
    }
}
```

Step 4: Comparison and logical (print true/false — no `if`)

```
public class ComparePrint {
    public static void main(String[] args) {
        int marks = 78;
        System.out.println(marks >= 35);
        System.out.println(marks >= 90);
        System.out.println((marks >= 35) && (marks < 90));
    }
}
```

Expected Output:

```
true
false
true
```

Flow Control will *use* these booleans in `if`. Today, only print them.

Store paid and due fees as 25000 each, then print whether they are equal and whether they are not equal.

```
public class FeeCheck {
    public static void main(String[] args) {
        double paid = 25000;
        double due = 25000;
        System.out.println(paid == due);
        System.out.println(paid != due);
    }
}
```

Store PIN `1234` and print whether its length is exactly 4.

```
public class PinLength {
    public static void main(String[] args) {
        String pin = "1234";
        System.out.println(pin.length() == 4);
    }
}
```

Step 5: Scanner input

```
import java.util.Scanner;

public class GreetScanner {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        String name = sc.nextLine();
        System.out.println("Hello, " + name);
        sc.close();
    }
}
```

Note:
- Wrong: Mixing `nextInt()` then `nextLine()` without care — leftover newline bites later.
- Prefer: `nextLine()` + `Integer.parseInt` / `Double.parseDouble` this week (simple and consistent).

Read a city name with `Scanner` and print `You typed:` followed by what the user entered.

```
import java.util.Scanner;

public class ReadCity {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("City: ");
        String city = sc.nextLine();
        System.out.println("You typed: " + city);
        sc.close();
    }
}
```

Read quantity as a line of text, convert it with `Integer.parseInt`, and print `Qty is` plus the number.

```
import java.util.Scanner;

public class ReadQty {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Qty: ");
        int qty = Integer.parseInt(sc.nextLine());
        System.out.println("Qty is " + qty);
        sc.close();
    }
}
```

Step 6: Casting and wrappers lite

```
public class CastingLite {
    public static void main(String[] args) {
        String raw = "78.5";
        double marks = Double.parseDouble(raw);
        int truncated = (int) marks;
        Integer boxed = Integer.valueOf(42);

        System.out.println(marks);
```

```
        System.out.println(truncated);
        System.out.println(boxed);
    }
}
```

Expected Output:

```
78.5
78
42
```

Wrappers (`Integer`, `Double`) are objects around primitives. You need them later for collections. This week: `parseInt` / `parseDouble` / `valueOf` is enough.

Interview: Autoboxing (`Integer x = 5`) exists — say "compiler wraps the primitive" if asked.

Parse roll `"2026001"` with `Integer.parseInt`, add 1, and print the next roll number.

```
public class ParseRoll {
    public static void main(String[] args) {
        int roll = Integer.parseInt("2026001");
        System.out.println(roll + 1);
    }
}
```

Expected Output:

```
2026002
```

Cast average `79.9` to `int` and print the truncated value.

```
public class TruncateMarks {
    public static void main(String[] args) {
        double avg = 79.9;
        System.out.println((int) avg);
    }
}
```

Expected Output:

```
79
```

Step 7: Formatted money (printf)

```
public class PrintfMoney {
    public static void main(String[] args) {
        double total = 105.0;
        System.out.printf("Total: ₹%.2f%n", total);
    }
}
```

Expected Output:

```
Total: ₹105.00
```

`%n` is a newline. `%.2f` is two decimal places.

Print average `77.5` with `printf` so it shows two decimal places.

```
public class AvgFormat {
    public static void main(String[] args) {
        double avg = 77.5;
        System.out.printf("Average: %.2f%n", avg);
    }
}
```

Print a Swiggy GST line of ₹4.00 using `printf` with two decimal places.

```
public class GstLine {
    public static void main(String[] args) {
        double gst = 4.0;
        System.out.printf("GST: ₹%.2f%n", gst);
    }
}
```

Step 8: Linear canteen calculator (annotated)

```
import java.util.Scanner;

public class CanteenCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double gstRate = 0.05;

        System.out.print("Price: ");
        double price = Double.parseDouble(sc.nextLine());
        System.out.print("Qty: ");
        int qty = Integer.parseInt(sc.nextLine());

        double subtotal = price * qty;
        double total = Math.round(subtotal * (1 + gstRate) * 100.0) / 100.0;

        System.out.printf("Total: ₹%.2f%n", total);
        sc.close();
    }
}
```

Sample run (input `40` then `2`):

Expected Output:

```
Total: ₹84.00
```

Lab kit: 15_Lab_Assets/03_canteen_calculator. Casting bugs: 04_casting_bugs.

Practice Exercises

Basic

1. Declare `int age`, `double heightCm`, `boolean isStudent` and print all three.

Model answer:

```
public class BasicVars {
    public static void main(String[] args) {
        int age = 21;
        double heightCm = 162.5;
        boolean isStudent = true;
        System.out.println(age);
        System.out.println(heightCm);
        System.out.println(isStudent);
    }
}
```

2. Compute `10 + 3`, `10 - 3`, `10 * 3`, `10 / 3`, `10 / 3.0`, `10 % 3`.

Model answer: `13`, `7`, `30`, `3`, `3.333…`, `1`. Integer `/` truncates.

3. Read a name with `Scanner` and print `Namaste, <name>`.

Model answer: `String name = sc.nextLine();` then `System.out.println("Namaste, " + name);`

4. Convert `"49.99"` to `double` and print it.

Model answer: `double p = Double.parseDouble("49.99");`

Intermediate

1. Read price (`double`) and qty (`int`); print subtotal and 5% GST total with two decimals. Linear only.

Model answer: Same shape as `CanteenCalculator` above. Total = `price * qty * 1.05`.

2. Fix: `int marks = Integer.parseInt("78.5");`

Model answer: Use `Double.parseDouble("78.5")` (or `"78"` with `parseInt`). `parseInt` does not accept a decimal point.

3. Why is `5 / 2` equal to `2` in Java?

Model answer: Both operands are `int`, so the result type is `int`. The fraction is discarded. Write `5 / 2.0` or `(double) 5 / 2`.

4. Read three marks as `double` and print the average with `printf` `%.2f`. No loops.

Model answer: Three `parseDouble` lines; `avg = (m1 + m2 + m3) / 3.0;` then `System.out.printf("Average: %.2f%n", avg);`

Interview-Oriented

1. Primitive vs reference type — one sentence.

Model answer: Primitives (`int`, `double`) hold the value in the variable; `String` is a reference to an object in memory (details in Strings / OOP).

2. What does `==` do on two `int` values?

Model answer: Compares the numeric values. (`==` on two `String` objects is a trap — Strings week.)

3. Why close `Scanner` on `System.in` in tiny demos?

Model answer: Good habit to release the resource. In tiny `main` programs it is optional; still show `close()` so later try-with-resources feels familiar. Do **not** use try-with-resources until Exceptions week.

4. Wrapper example in one line.

Model answer: `Integer n = Integer.valueOf(10);` boxes `10`. Collections later store wrappers, not raw `int`.

Mini-Project (Assignment): Swiggy Canteen Calculator

Lab kits: 03_canteen_calculator · 04_casting_bugs

What to build

1. `CanteenCalculator.java` — `float` / `double` price, `int` qty, 5% GST, `Total: ₹xx.xx`
2. Fix the casting bugs kit (`78.5` must work)
3. Push the calculator to a GitHub repo (even if Git week is later — account + zip is OK this week)

Stretch

- **Later — Flow Control:** reject negative qty with `if`.
- Print subtotal and GST as separate lines.

Done when

- [ ] Linear `main` only (no `if` / loops / extra methods)
- [ ] GST math is percent, not `+ 0.05` rupees
- [ ] Two-decimal money print
- [ ] Casting kit uses `double` for decimal marks

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

Q1. What is a variable in Java?
A: A named, typed storage location. `int roll = 101;` reserves an `int` box labelled `roll`.

Q2. Why is `Scanner` input converted with `parseInt` / `parseDouble`?
A: Keyboard text is `String`. Math needs numbers. Conversion makes `"40"` into `40`.

Q3. Integer division vs floating division?
A: `int/int` truncates. Use a `double` operand when you need a decimal result (marks average, rupees).

Q4. Widening vs narrowing?
A: `int` to `double` is safe (widening). `double` to `int` needs a cast and **truncates** (narrowing).

Q5. Name eight primitive types.
A: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`. This course lives mostly on `int`, `double`, `boolean`, `char`.